

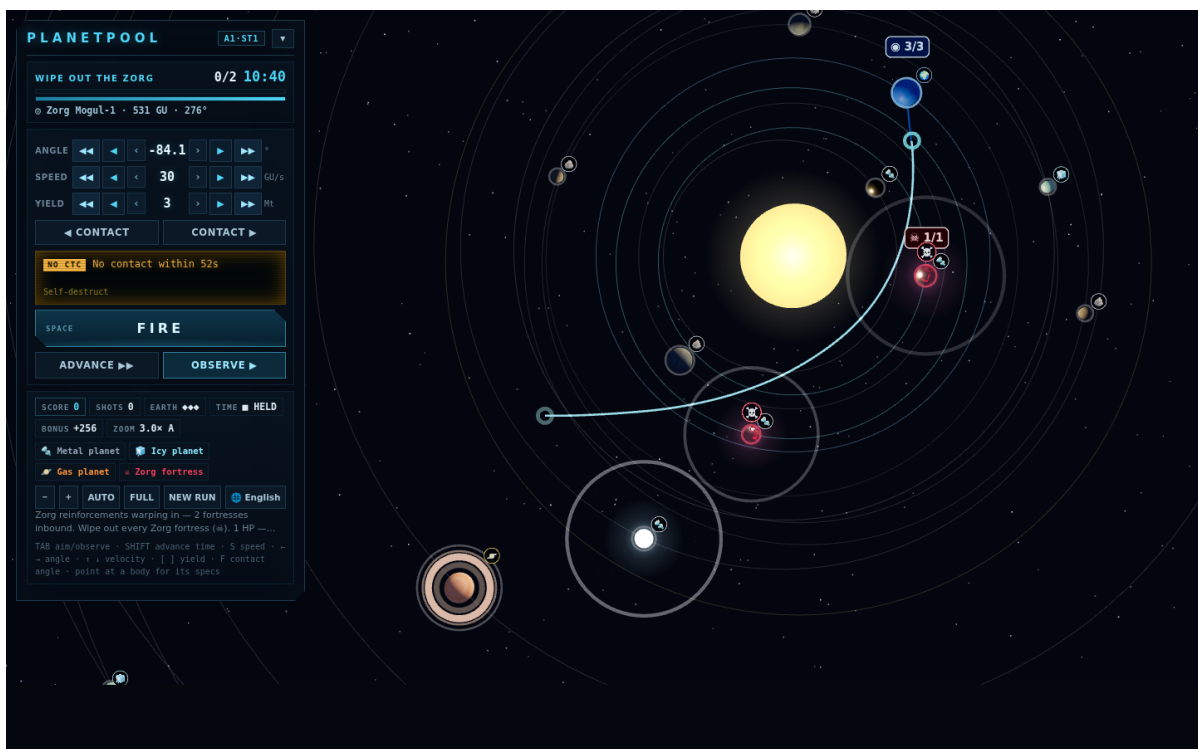
PLANETPOOL × Codex

행성당구 로그라이크 — AI와 함께 만든 과정 기록

세 줄 요약

- ① 핵미사일을 중력 스윙바이로 휘어 쏘는 당구 게임 — 화면의 모든 행성이 실제 N체 중력 시뮬레이션으로 움직입니다.
- ② 궤도역학·수치 적분 같은 전문 지식은 Codex가 채웠고, 재미의 판정과 모든 방향 결정은 사람이 했습니다.
- ③ 이미지·오디오 등 외부 에셋 파일이 하나도 없습니다 — 그래픽은 three.js로 그리고, 효과음도 전부 코드로 합성합니다.

PLANETPOOL은 핵미사일을 행성의 중력에 태워 휘어 쏘는 우주 당구 로그라이크입니다. 행성은 고정된 장애물이 아닙니다. 모든 행성이 뉴턴 중력으로 서로를 끌어당기며 궤도를 바꾸는 N체 시뮬레이션 위에서, 직선으로 쏘면 지고 궤적을 휘어야 이깁니다 — 궤도역학이라는 전문 지식 자체가 게임의 정체성인 프로젝트입니다.



실제 플레이 화면. 예측선(하늘색)이 태양 중력에 붙잡혀 휘어 들어간다 — 직선 조준으로는 닿지 않는 판이다. 왼쪽은 LCD풍 사격 통제 패널(각도·속도·탄두 위력·교신 기록).

기술 스택	바닐라 JavaScript(ES 모듈) + three.js · 효과음은 ZzFX로 코드 합성 · 빌드 도구 없이 정적 호스팅만으로 배포
물리	뉴턴 중력 + 소프트닝, KDK 리프로그(심플렉틱) 적분, 스윙바이 판정, 임펄스/파쇄 모델
제작 방식	'사람이 방향 제시 → Codex가 구현 → 직접 플레이로 검증'을 짧은 주기로 반복 · 5개 언어 지원(한국어·영어·일본어·중국어 간체/번체)

직접 확인하기 게임 폴더에서 `npm run serve`를 실행한 뒤 브라우저로 `localhost:4173`에 접속하면 바로 플레이할 수 있습니다(설치할 의존성 없음). 인트로 없이 시작하려면 `?nointro=1`, 같은 판을 다시 보려면 `?seed=숫자`를 주소에 붙입니다.

1. Codex를 어디에 사용했나요?

개발 전 과정에서 역할을 나누어 썼습니다. 사람이 방향을 정하면, Codex가 지식을 조사하고 코드로 구현했습니다.

영역	Codex가 한 일
기술 선정	라이브러리와 배포 방식을 탐색·비교해 선정 — three.js 로컬 번들과 빌드 없는 정적 배포를 택한 근거 제시
도메인 지식	중력 계수($\mu=GM$), 탈출 속도, 행성 질량, 공전 속도, N체 문제의 수치 시뮬레이션 방법 등 천문학 지식을 검색·취합해 게임 수식으로 정리
아키텍처	주어진 제약(빌드 도구 없는 바닐라 JS) 안에서 알맞은 프로그래밍 패턴과 코드 구조를 설계하고 코드를 작성
오디오 구현	주요 동작별 효과음(발사음·레이저·폭발·교신 잡음)을 오디오 파일 없이 ZzFX 합성 파라미터로 설계·구현
UI 다듬기	결과물을 직접 실행해 보며 UI 문구·배치·색·크기를 반복해서 조정
시퀀스 구현	인트로·튜토리얼처럼 깊은 논리 없이 손이 많이 가는 연출 시퀀스를 대량 작성
리팩토링	폴더 구조 정리와 코드 리팩토링(core/game/render/ui/i18n 분리)
다국어	전체 문안 번역과 5개 언어 대응 체계 구축

2. 어떤 기능을 구현했나요?

- N체 물리 엔진 — 태양과 모든 행성 사이의 상호 중력(소프트닝 포함)을 매 프레임 계산. 시간이 오래 지나도 궤도가 무너지지 않도록 에너지를 보존하는 KDK 리프프로그 적분기 사용
- 스윙바이 게임플레이 — 조준·발사 시스템, 수천 스텝을 미리 계산하는 궤적 예측선, 근접 통과 거리와 굴절각에 따른 스윙바이 판정과 점수 배율
- 로그라이크 진행 — 판마다 다시 차오르는 성계, 카이퍼 벨트, 워프 증원, 조르그 레이저·모함 등 라운드가 진행될수록 커지는 위협
- 동작별 효과음 — 발사음, 레이저, 폭발, 교신 잡음까지 주요 동작 하나하나를 ZzFX로 실시간 합성(미리 합성해 캐시, 마스터 컴프레서로 음량 정리)
- 연출과 온보딩 — 오프닝 컷신, 단계별 튜토리얼, 3D 발사기 연출, 필요한 설명을 필요한 판에만 보여 주는 안내 체계
- UI/UX — LCD풍 사격 통제 HUD, 궤도 표시, 관측 배속, 언어 선택을 포함한 5개 언어 인터페이스

3. 어떤 문제를 해결했나요?

- 궤도가 무너지는 문제 — 널리 쓰이는 RK4 적분은 에너지 오차가 쌓여 몇 분 뒤 궤도가 새어 나갑니다. Codex가 천체 시뮬레이션의 표준인 심플렉틱(리프프로그) 적분을 제안하고 구현해 장시간 안정성을 확보했습니다.
- 예측선 성능 저하 — 예측선을 한 번 그릴 때마다 수천 스텝 × 천체 수만큼 객체가 새로 생성되어, 행성이 20개로 늘자 프레임이 떨어졌습니다. 가속도 버퍼를 재사용해 불필요한 메모리 할당을 없앴습니다.
- 웹 오디오의 함정 — 소리 재생을 막는 브라우저 자동 재생 정책은 첫 사용자 입력(CLICK TO START)에서 오디오를 깨워 해결했습니다. 음량이 급변할 때 나는 잡음은 최소 5ms의 완만한 음량 변화로, 같은 효과음이 한꺼번에 몰려 뭉개지는 문제는 30ms 안 중복 재생 억제와 마스터 컴프레서로 잡았고, 효과음을 미리 합성해 두어 발사 순간의 프레임 저하도 막았습니다.
- 네트워크 없이도 도는 배포 — CDN이 막힌 환경에서도 로딩이 멈추지 않도록 라이브러리를 로컬 번들로 내장하고, 로딩에 실패하면 원인을 화면에 그대로 알려 주는 오류 처리를 붙였습니다.
- "당구인데 볼링이 되는" 문제 — 직선으로 쏘아 이기는 판이 나오지 않도록, 기획서의 차폐도·당구감 계수 같은 규칙을 수식으로 만들어 목표 경험을 코드로 강제했습니다.

4. 사람이 직접 결정한 부분은 무엇인가요?

- 게임의 정체성 — "직사는 실패다, 궤적이 휘어야 한다, 판은 살아 있다"라는 목표 경험의 정의와 우선순위
- AI에게 일을 시키는 방식의 설계 — 기획서의 수식 20개를 "매 프레임 도는 것 / 판을 만들 때 한 번만 도는 것 / 코드에는 안 들어가고 밸런스 조정에만 쓰는 것" 세 갈래로 사람이 직접 분류해, AI가 판단을 보태지 않고 그대로 구현하면 되는 명세로 만들었습니다
- 재미의 판정 — 매번 결과물을 직접 플레이하며 밸런스(체력, 대응 시간, 배속, 판돈 구성)를 감각으로 조정하고 다음 방향을 지시
- 취사선택 — Codex가 정리한 수식·설계안 가운데 무엇을 채택하고 무엇을 버릴지 최종 결정(예: 지나친 천문학적 정밀도는 화면에서 구분되지 않으므로 단순화)

5. 만약 AI가 없었다면 — 이 게임에서 AI를 쓴 의미

행성당구는 아이디어만으로는 만들 수 없는 장르입니다. 게임의 재미 자체가 진짜 궤도역학 위에 서 있기 때문입니다. 전공자가 아닌 개인이 이 게임을 만들려면 다음 지식을 스스로 익혀야 합니다.

- 중력 계수 $\mu=GM$ 과 질량·반지름·내구도의 파생 관계, 탈출 속도와 공전 속도의 계산
- N체 문제는 해석적 해가 없다는 사실과 수치 적분기의 선택(왜 RK4가 아니라 심플렉틱인가)
- 스윙바이(중력 어시스트)의 원리와 굴절각, 안정된 궤도 배치를 위한 힐 반경 같은 개념

이는 교양서 몇 권이 아니라 항공우주역학 교재와 수치 해석 자료를 오가며 몇 주에서 몇 달을 들여야 하는 학습량입니다. 지식을 얻어도 그것을 초당 60프레임으로 도는 게임 코드로 옮기는 일은 또 다른 벽입니다. Codex는 지식의 벽과 구현의 벽을 동시에 낮췄습니다 — 필요한 전문 지식을 그 자리에서 검색·취합해 주었고, 그 지식을 곧바로 검증할 수 있는 코드로 만들어 주었습니다. 사람은 "무엇이 재미있는가"에만 집중하고 AI가 "어떻게 물리적으로 옮겨 만드는가"를 채우는 분업 — 그 덕에 일반 개발자 혼자서는 시작조차 어려웠을 실제 N체 시뮬레이션 게임이 완성되었습니다.

"AI는 코드를 대신 써 준 것이 아니라, 이 게임이 존재할 수 있도록 지식의 문턱을 없애 주었다."